SIMD-CoreX

Design and SimulationofaFour-Lane SIMDArithmetic Datapath Using Verilog HDL

Project Report

Submitted by

Vibhav Teotia

Roll Number: 2400320310203

ABES Engineering College

Department: Electronics and Communication Engineering

Tool: Xilinx Vivado 2018.3 | HDL: Verilog | Verification: Behavioral Simulation

Abstract

SIMD-CoreX isa Verilog HDL design that demonstrates Single Instruction, Multiple Data (SIMD)

processing using four signed 8-bit lanes. A 2-bit opcode selects vector addition, vector subtraction, lane-wise multiplication, or a dot-product operation. The design captures input operands and the opcode in a first clocked stage, then registers the selected result in a second stage. The included testbench checks the four operations and the uploaded Vivado simulation log records that all four test cases passed.

1. Objectives

- Implement afour-laneSIMD arithmetic datapath in synthesizable Verilog HDL.
- Support vector addition, vector subtraction, lane-wise multiplication, and dot product.
- Use valid signalling and clocked registers to control data capture and output timing.
- Verify expected results using a self-checking Vivado testbench.

2. Tools and Project Files

Thesupplied archivecontainsa Vivado project file (MySIMDCOREX.xpr), the main RTL module (simdcoreX.v), an FPGA-oriented wrapper (simd_fpgatop.v), the testbench (tb_simdcoreX.v), a waveform configuration file, simulation output, and implementation reports. The project metadata identifies the target part as xa7a15tcsg324-2I. The project has been simulated; the user reports implementation completed. No physical FPGA board test is claimed.

3. Architecture

Thetop-level module receives two 32-bit packed operands A and B, a 2-bit opcode, a valid input, clock, and active-low reset. Each packed operand is interpreted as four signed 8-bit lanes. Stage 1 registers the operands and opcode when in_valid is asserted and pipelines the valid flag. Combinational lane arithmetic forms four 8-bit additions/subtractions and four signed 16-bit products. For DOT, the four products are sign-extended and added into a signed 20-bit sum. Stage 2 registers vector_result, dot_result, and out_valid.

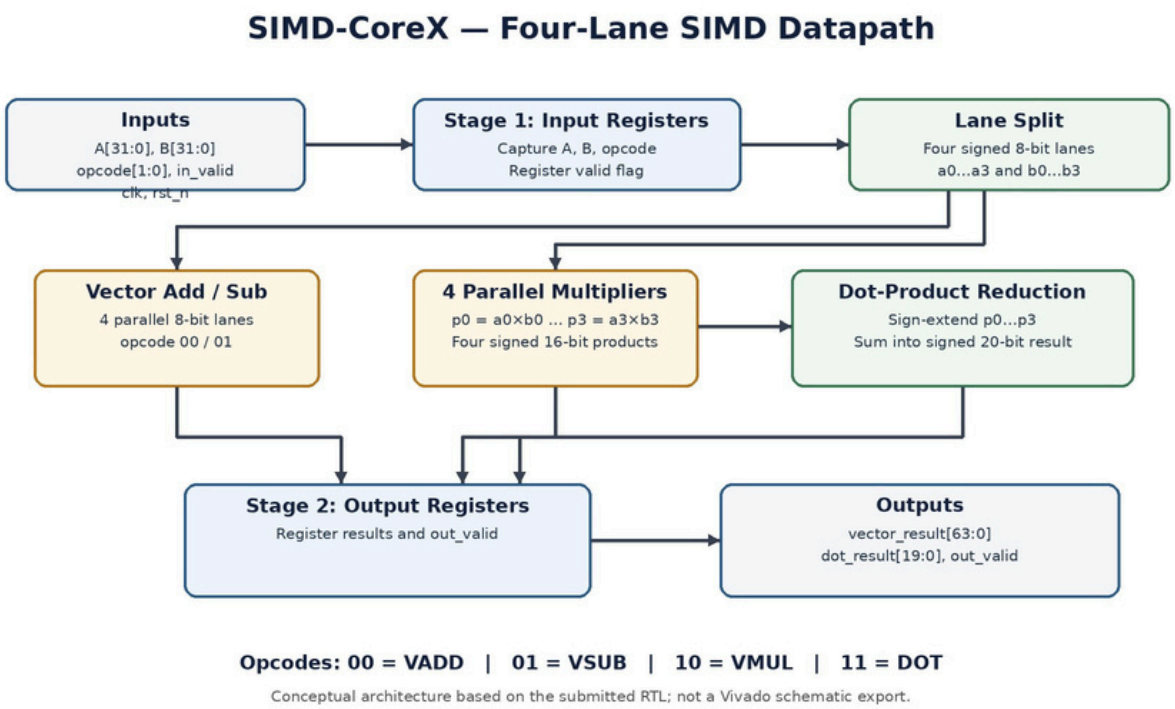


Figure 1. Conceptual architecture derived from the submitted RTL; it is not a screenshot of Vivado's schematic.

4. Opcode Table

Opcode	Operation	Result behavior
00	VADD	Four 8-bit lane sums packed into vector_result
01	VSUB	Four 8-bit lane differences packed into vector_result
10	VMUL	Four signed 16-bit products packed into vector_result
11	DOT	Sum of four signed lane products in dot_result; vector_result is zero

5. Verification Method

Theself-checking testbench applies a synchronous active-low reset, then drives each operation with packed operands A = 32'h04030201 and B = 32'h0E030201. The expected lane values are A = {4,3,2,1} and B = {14,3,2,1}, where lane 0 is the least-significant byte. The testbench checks out_valid, vector_result, and dot_result for each operation and terminates with a fatal error if an expected value does not match.

6. Simulation Results

Opcode	Operation	Expected vector_result	Expected dot_result
00	VADD	0000000012060402	0
01	VSUB	00000000F6000000	0
10	VMUL	0038000900040001	0

Opcode	Operation	Expected vector_result	Expected dot_result
11	DOT	0000000000000000	70

The uploaded simulation log from Vivado Simulator 2018.3 reports PASS for opcodes 00, 01, 10, and 11, followed by “ALL SIMD-COREX TESTS PASSED”. The DOT test computes 1×1 + 2×2 + 3×3 + 4×14 = 70. The log is included separately in the submission package.

7. Implementation Notes and Limitations

The supplied project archive includes routed implementation reports. The route-status report records 693 routable nets fully routed and zero nets with routing errors. This is a report from the uploaded project files, not an independent rerun of Vivado in this environment. The design has not been verified on a physical FPGA board according to the information supplied by the student. The wrapper simd_fpgatop.v fixes the opcode to DOT and maps the low eight bits of dot_result to LEDs, but successful board operation depends on a compatible board, valid pin constraints, and hardware testing.

8. Conclusion

The submitted SIMD-CoreX RTL implements four-lane signed arithmetic with registered inputs and outputs. The provided behavioral simulation log demonstrates passing checks for vector addition, vector subtraction, lane-wise multiplication, and a dot-product test returning 70. The project can be further developed with additional test vectors, timing analysis, and hardware validation on a compatible FPGA board.

Appendix: Files included

MySIMDCOREX_Original_Vivado_Project.zip — original uploaded project archive.
RTL_and_Testbench/simdcoreX.v — top-level SIMD datapath.
RTL_and_Testbench/simd_fpgatop.v — FPGA-oriented wrapper.
RTL_and_Testbench/tb_simdcoreX.v — self-checking testbench.
Documentation/SIMD_CoreX_Architecture.png — conceptual architecture diagram.
Documentation/Vivado_Simulation_Log.txt — simulation transcript from the uploaded project.

Before submitting, verify the department wording and any required cover-page format with your course instructions. Add a genuine Vivado waveform screenshot if the portal or faculty explicitly requires a screenshot; the simulation transcript is included but is not a waveform screenshot.

SIMD-CoreX

Project Evidence and Source Code Appendices

Submitted by: Vibhav Teotia **Roll Number:** 2400320310203

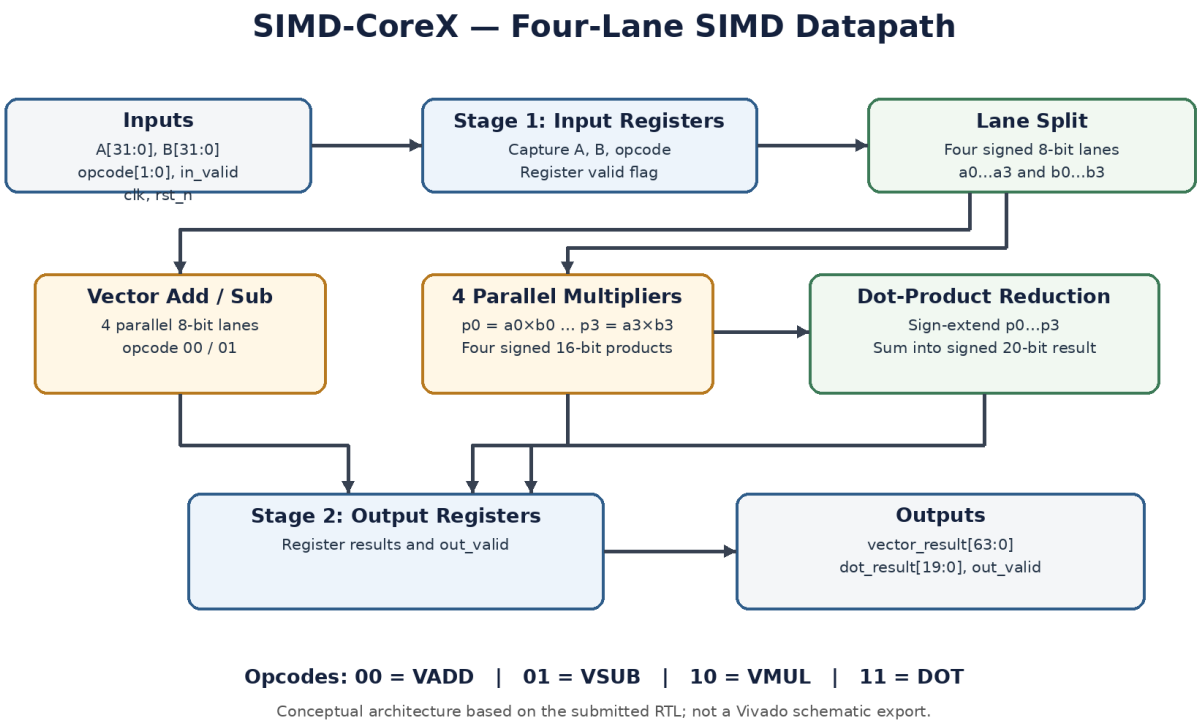
Department: ELECTRONICS AND COMMUNICATION ENGINEERING

College: ABES Engineering College

This section supplements the original report with the submitted architecture image, Vivado waveform screenshot, source RTL, wrapper module, testbench, and simulation log. The evidence documents simulation and project files; it does not claim testing on a physical FPGA board.

Appendix A — Architecture Diagram

Conceptual four-lane SIMD datapath architecture.



Appendix B — Vivado Behavioral Simulation Waveform

The supplied waveform shows the clock, reset, valid signal, opcode, operands, vector result, dot result, and output-valid signal. The displayed dot_result value is 70 for opcode 3 (DOT).

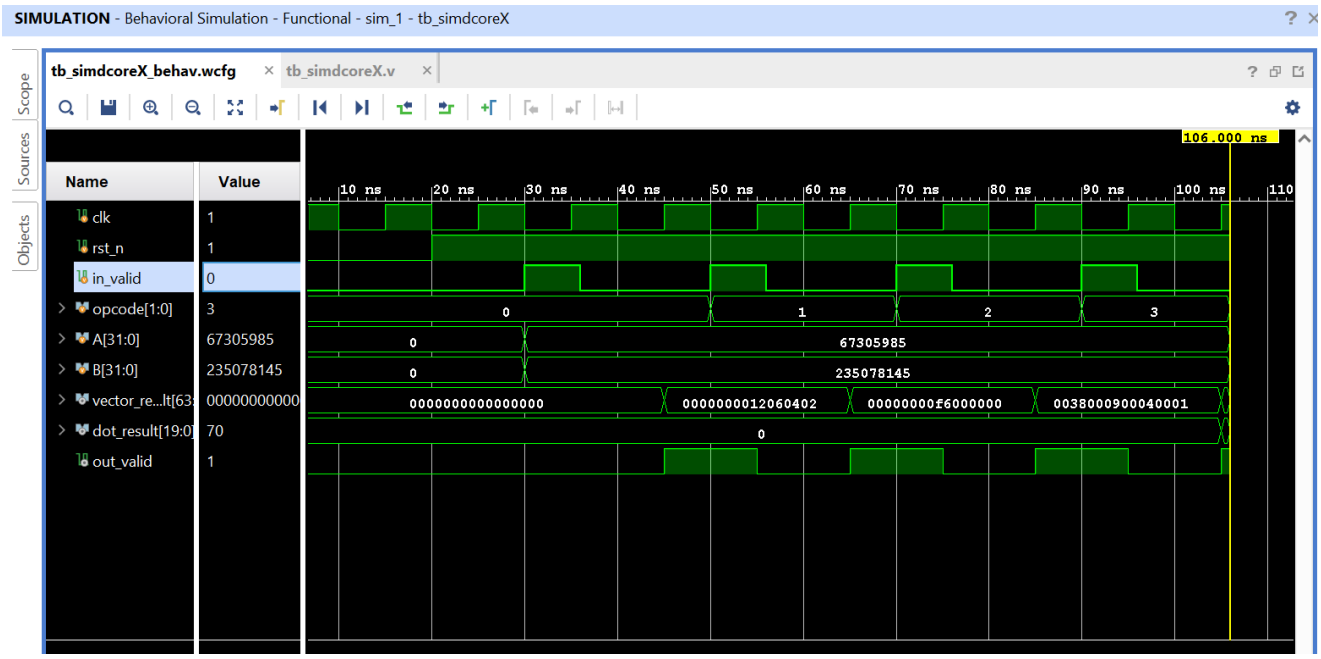


Figure B1. User-provided Vivado behavioral simulation waveform screenshot.

Appendix C — Main RTL: simdcoreX.v

Source code reproduced from the submitted ZIP archive.

```
`timescale 1ns / 1ps



module simdcoreX(
    input wire clk,
    input wire rst_n,
    input wire in_valid,
    input wire [1:0] opcode,
    input wire [31:0] A,
    input wire [31:0] B,

    output reg [63:0] vector_result,
    output reg signed [19:0] dot_result,
    output reg out_valid
);

    // Opcode definitions
    localparam VADD = 2'b00;
    localparam VSUB = 2'b01;
    localparam VMUL = 2'b10;
    localparam DOT  = 2'b11;

    // Stage 1: Input registers
    reg [31:0] A_q, B_q;
    reg [1:0] opcode_q;
    reg valid_q;

    // Signed input lanes
    wire signed [7:0] a0 = A_q[7:0];
    wire signed [7:0] a1 = A_q[15:8];
    wire signed [7:0] a2 = A_q[23:16];
    wire signed [7:0] a3 = A_q[31:24];

    wire signed [7:0] b0 = B_q[7:0];
    wire signed [7:0] b1 = B_q[15:8];
    wire signed [7:0] b2 = B_q[23:16];
    wire signed [7:0] b3 = B_q[31:24];

    // Four parallel signed multipliers
    wire signed [15:0] p0 = a0 * b0;
    wire signed [15:0] p1 = a1 * b1;
    wire signed [15:0] p2 = a2 * b2;
    wire signed [15:0] p3 = a3 * b3;

    // Signed 20-bit dot-product reduction
    wire signed [19:0] dot_sum =
          {{4{p0[15]}}, p0}
        + {{4{p1[15]}}, p1}
        + {{4{p2[15]}}, p2}
        + {{4{p3[15]}}, p3};

    // Stage 1: Capture input operands and opcode
    always @(posedge clk) begin
        if (!rst_n) begin
            A_q      <= 32'b0;
            B_q      <= 32'b0;
            opcode_q <= VADD;
            valid_q  <= 1'b0;
        end
        else begin
            valid_q <= in_valid;

            if (in_valid) begin
                A_q      <= A;
                B_q      <= B;
                opcode_q <= opcode;
            end
        end
    end

    // Stage 2: Execute and register outputs
    always @(posedge clk) begin
        if (!rst_n) begin
            vector_result <= 64'b0;
```

Appendix C — Main RTL: simdcoreX.v (continued)

```
            dot_result    <= 20'sd0;
            out_valid     <= 1'b0;
        end
        else begin
            out_valid <= valid_q;

            if (valid_q) begin
                case (opcode_q)

                    VADD: begin
                        vector_result <= {
                            (a3 + b3),
                            (a2 + b2),
                            (a1 + b1),
                            (a0 + b0)
                        };
                        dot_result <= 20'sd0;
                    end

                    VSUB: begin
                        vector_result <= {
                            (a3 - b3),
                            (a2 - b2),
                            (a1 - b1),
                            (a0 - b0)
                        };
                        dot_result <= 20'sd0;
                    end

                    VMUL: begin
                        vector_result <= {
                            p3, p2, p1, p0
                        };
                        dot_result <= 20'sd0;
                    end

                    DOT: begin
                        vector_result <= 64'b0;
                        dot_result <= dot_sum;
                    end

                    default: begin
                        vector_result <= 64'b0;
                        dot_result <= 20'sd0;
                    end

                endcase
            end
        end
    end

endmodule
```

Appendix D — FPGA Wrapper: simd_fpgatop.v

Source code reproduced from the submitted ZIP archive.

```
`timescale 1ns / 1ps



module simd_fpgatop(
    input  wire       clk,
    input  wire       rst_n,
    output wire [7:0] led,
    output wire       valid_led
);

    wire [63:0] vector_result;
    wire signed [19:0] dot_result;
    wire out_valid;

    simdcoreX uut (
        .clk(clk),
        .rst_n(rst_n),
        .in_valid(1'b1),
        .opcode(2'b11),       // DOT operation
        .A(32'h04030201),     // Lanes: 1, 2, 3, 4
        .B(32'h0E030201),     // Lanes: 1, 2, 3, 14

        .vector_result(vector_result),
        .dot_result(dot_result),
        .out_valid(out_valid)
    );

    assign led       = dot_result[7:0];
    assign valid_led = out_valid;

endmodule
```

Appendix E — Testbench: tb_simdcoreX.v

Source code reproduced from the submitted ZIP archive.

```
`timescale 1ns / 1ps


module tb_simdcoreX;

    reg clk;
    reg rst_n;
    reg in_valid;
    reg [1:0] opcode;
    reg [31:0] A;
    reg [31:0] B;

    wire [63:0] vector_result;
    wire signed [19:0] dot_result;
    wire out_valid;

    // Device Under Test
    simdcoreX uut (
        .clk(clk),
        .rst_n(rst_n),
        .in_valid(in_valid),
        .opcode(opcode),
        .A(A),
        .B(B),
        .vector_result(vector_result),
        .dot_result(dot_result),
        .out_valid(out_valid)
    );

    // 10 ns clock period
    initial clk = 1'b0;
    always #5 clk = ~clk;

    // Apply stimulus and verify outputs
    task check_operation;
        input [1:0] op;
        input [31:0] input_A;
        input [31:0] input_B;
        input [63:0] expected_vector;
        input signed [19:0] expected_dot;

        begin
            // Drive inputs away from the rising clock edge
            @(negedge clk);
            opcode   = op;
            A        = input_A;
            B        = input_B;
            in_valid = 1'b1;

            // Stage 1: Capture input
            @(posedge clk);
            #1;
            in_valid = 1'b0;

            // Stage 2: Execute and register output
            @(posedge clk);
            #1;

            if (out_valid !== 1'b1) begin
                $display("FAIL: opcode=%b, out_valid=%b",
                         op, out_valid);
                $fatal(1, "Output valid check failed");
            end

            if ((vector_result !== expected_vector) ||
                (dot_result !== expected_dot)) begin

                $display("FAIL: opcode=%b", op);
                $display("Expected vector = %h", expected_vector);
                $display("Actual vector   = %h", vector_result);
                $display("Expected dot    = %0d", expected_dot);
                $display("Actual dot      = %0d", dot_result);

                $fatal(1, "SIMD output mismatch");
```

Appendix E — Testbench: tb_simdcoreX.v (continued)

```
            end
            else begin
                $display("PASS: opcode=%b", op);
                $display("Vector result = %h", vector_result);
                $display("Dot result    = %0d", dot_result);
            end
        end
    endtask

    initial begin
        // Initialize inputs
        clk      = 1'b0;
        rst_n    = 1'b0;
        in_valid = 1'b0;
        opcode   = 2'b00;
        A        = 32'b0;
        B        = 32'b0;

        // Apply synchronous active-low reset
        repeat (2) @(posedge clk);
        @(negedge clk);
        rst_n = 1'b1;

        // Lane 3 = 4, lane 2 = 3,
        // lane 1 = 2, lane 0 = 1
        // A = {4,3,2,1}
        // B = {14,3,2,1}

        // Test 1: VADD
        check_operation(
            2'b00,
            32'h04030201,
            32'h0E030201,
            64'h0000000012060402,
            20'sd0
        );

        // Test 2: VSUB
        check_operation(
            2'b01,
            32'h04030201,
            32'h0E030201,
            64'h00000000F6000000,
            20'sd0
        );

        // Test 3: VMUL
        check_operation(
            2'b10,
            32'h04030201,
            32'h0E030201,
            64'h0038000900040001,
            20'sd0
        );

        // Test 4: DOT
        // 1*1 + 2*2 + 3*3 + 4*14 = 70
        check_operation(
            2'b11,
            32'h04030201,
            32'h0E030201,
            64'h0000000000000000,
            20'sd70
        );

        $display("================================");
        $display("ALL SIMD-COREX TESTS PASSED");
        $display("================================");

        $finish;
    end

endmodule
```

Appendix F — Simulation Log

Submitted simulation log text:

```
Vivado Simulator 2018.3
Time resolution is 1 ps
PASS: opcode=00
Vector result = 0000000012060402
Dot result    = 0
PASS: opcode=01
Vector result = 00000000f6000000
Dot result    = 0
PASS: opcode=10
Vector result = 0038000900040001
Dot result    = 0
PASS: opcode=11
Vector result = 0000000000000000
Dot result    = 70
================================
ALL SIMD-COREX TESTS PASSED
================================
$finish called at time : 106 ns : File "D:/My Codes/MySIMDCOREX/MySIMDCOREX.srcs/sim_1/new/tb_simdcoreX.v" Line 145
```